

Projeto guiado: recorte de imagem com Flask e OpenCV

Material do aluno - Backend Python + Frontend HTML

Este guia foi feito para você construir o sistema em etapas. A regra principal é simples: primeiro tente resolver, depois compare com a referência visual. O professor deve ser chamado quando você souber dizer em qual etapa travou e qual mensagem de erro apareceu.

Resultado esperado

Ao final, você terá uma aplicação web pequena que:

- abre uma página HTML pelo Flask;
- recebe dados de um formulário;
- recebe uma imagem enviada pelo navegador;
- recebe coordenadas de recorte;
- usa OpenCV para cortar a imagem;
- salva o recorte em uma pasta do projeto;
- mostra no navegador se a operação funcionou ou falhou.

Como trabalhar

- Faça um passo por vez.
- Teste sempre antes de ir para o próximo passo.
- Digite o código pensando no papel de cada linha.
- Quando aparecer erro, leia a mensagem antes de chamar o professor.
- Não comece pelo OpenCV. Primeiro confirme que o navegador conversa com o Flask.

Estrutura do projeto

Crie uma pasta para o projeto. Dentro dela, a estrutura final deve ficar assim:

```
1  recorte-imagem/  
2      recortes.py  
3      templates/  
4          index.html  
5      uploads/  
6      recortes/
```

O Flask procura arquivos HTML dentro da pasta `templates`. Por isso, neste guia o arquivo da página fica em `templates/index.html`.

Preparação

No terminal, dentro da pasta do projeto, crie um ambiente virtual e instale as bibliotecas:

```
1  python3 -m venv .venv  
2  source .venv/bin/activate  
3  pip install flask opencv-python
```

Se estiver no Windows, a ativação pode ser:

```
1  .venv\Scripts\activate
```

Passo 1 - Colocar o Flask no ar

Desafio

Crie um servidor Flask com uma rota principal `/`. Essa rota deve devolver a página `index.html`.

Antes de olhar a referência, responda:

- Qual arquivo será executado pelo Python?
- Qual arquivo será entregue ao navegador?
- Qual função Python representa a rota `/`?

Referência visual do backend

```
1 from flask import Flask, render_template
2
3 app = Flask(__name__)
4
5
6 @app.route("/")
7 def index():
8     return render_template("index.html")
9
10
11 if __name__ == "__main__":
12     app.run(debug=True)
```

Referência visual do HTML

Crie `templates/index.html`:

```
1 <!DOCTYPE html>
2 <html lang="pt-BR">
3 <head>
4     <meta charset="UTF-8">
5     <title>Primeiro passo Flask</title>
6 </head>
7 <body>
8     <h1>Servidor Flask funcionando</h1>
9     <p>Se você está vendo esta página, o backend entregou este HTML.</p>
10 </body>
11 </html>
```

Teste

Execute:

```
1 python recortes.py
```

Abra no navegador:

```
1 http://127.0.0.1:5000
```

Checkpoint: se a página abriu, o Flask está funcionando e encontrou o template.

Passo 2 - Enviar um formulário simples

Desafio

Agora o HTML deve enviar um nome para o backend. O backend deve receber o campo e responder uma mensagem.

Você vai praticar:

- `method="POST"` no formulário;
- `action="/enviar"` apontando para a rota do Flask;
- `request.form.get(...)` para ler campos enviados pelo formulário.

Alteração no backend

Adicione `request` no import e crie uma rota nova:

```
1 from flask import Flask, render_template, request
2
3
4 @app.route("/enviar", methods=["POST"])
5 def enviar():
6     nome = request.form.get("nome", "").strip()
7
8     if not nome:
9         return "Você não digitou nenhum nome."
10
11     return f"Olá, {nome}. O backend recebeu seu formulário com sucesso."
```

Alteração no HTML

Troque o conteúdo do `body` por um formulário:

```
1 <h1>Passo 2 - Enviar formulário para o Flask</h1>
2
3 <form action="/enviar" method="POST">
4     <label for="nome">Digite seu nome:</label>
5     <input type="text" id="nome" name="nome" required>
6     <button type="submit">Enviar</button>
7 </form>
```

Teste

Reinicie o servidor, abra a página, digite um nome e envie.

Se aparecer erro 404, confira o `action` do formulário e o endereço da rota no Flask. Eles precisam combinar.

Passo 3 - Enviar uma imagem

Desafio

Troque o formulário de texto por um formulário de upload. O backend deve salvar o arquivo recebido na pasta `uploads`.

Você vai praticar:

- `enctype="multipart/form-data";`
- `request.files`;
- `secure\_filename`;
- criação de pastas com `os.makedirs`;
- salvamento de arquivo enviado pelo navegador.

Backend: preparação da pasta

```
1 from flask import Flask, render_template, request
2 from werkzeug.utils import secure_filename
3 import os
4
5
6 UPLOAD_FOLDER = "uploads"
7 os.makedirs(UPLOAD_FOLDER, exist_ok=True)
```

Backend: rota de upload

```
1 @app.route("/upload", methods=["POST"])
2 def upload():
3     if "imagem" not in request.files:
4         return "Nenhuma imagem foi enviada."
5
6     arquivo = request.files["imagem"]
7
8     if arquivo.filename == "":
9         return "Nenhum arquivo foi selecionado."
10
11     nome_seguro = secure_filename(arquivo.filename)
12     caminho_arquivo = os.path.join(UPLOAD_FOLDER, nome_seguro)
13     arquivo.save(caminho_arquivo)
14
15     return f"Upload realizado com sucesso. Arquivo salvo em: {caminho_arquivo}"
```

HTML: formulário de imagem

```
1 <h1>Passo 3 - Enviar imagem para o Flask</h1>
2
3 <form action="/upload" method="POST" enctype="multipart/form-data">
4     <label for="imagem">Escolha uma imagem:</label>
5     <input type="file" id="imagem" name="imagem" accept="image/*" required>
6     <button type="submit">Enviar imagem</button>
7 </form>
```

Teste

Envie uma imagem qualquer. Depois confira se a pasta `uploads` foi criada e se o arquivo apareceu dentro dela.

Checkpoint: se o arquivo foi salvo, o navegador já está enviando arquivos para o Flask corretamente.

Passo 4 - Enviar imagem e coordenadas

Desafio

Agora o usuário deve enviar:

- uma imagem;
- a posição inicial X;
- a posição inicial Y;
- a largura do recorte;
- a altura do recorte;
- o nome do arquivo de saída.

Neste passo, o sistema ainda não precisa recortar. Ele só precisa receber, validar e mostrar os dados.

HTML: campos necessários

```
1  <h1>Passo 4 - Enviar imagem e coordenadas</h1>
2
3  <form action="/recortar" method="POST" enctype="multipart/form-data">
4      <div>
5          <label for="imagem">Escolha uma imagem:</label>
6          <input type="file" id="imagem" name="imagem" accept="image/*" required>
7      </div>
8
9      <div>
10         <label for="x">X:</label>
11         <input type="number" id="x" name="x" required>
12     </div>
13
14     <div>
15         <label for="y">Y:</label>
16         <input type="number" id="y" name="y" required>
17     </div>
18
19     <div>
20         <label for="w">Largura:</label>
21         <input type="number" id="w" name="w" required>
22     </div>
23
24     <div>
25         <label for="h">Altura:</label>
26         <input type="number" id="h" name="h" required>
27     </div>
28
29     <div>
30         <label for="nome_saida">Nome do novo arquivo:</label>
31         <input type="text" id="nome_saida" name="nome_saida" required>
32     </div>
33
34     <button type="submit">Enviar dados</button>
35 </form>
```

Backend: leitura dos campos

```

1  @app.route("/recortar", methods=["POST"])
2  def recortar():
3      if "imagem" not in request.files:
4          return "Nenhuma imagem foi enviada."
5
6      arquivo = request.files["imagem"]
7      nome_saida = request.form.get("nome_saida", "").strip()
8      x = request.form.get("x", "").strip()
9      y = request.form.get("y", "").strip()
10     w = request.form.get("w", "").strip()
11     h = request.form.get("h", "").strip()
12
13     if arquivo.filename == "":
14         return "Nenhum arquivo foi selecionado."
15
16     if not nome_saida:
17         return "O nome do arquivo de saída não foi informado."
18
19     if not x or not y or not w or not h:
20         return "Os dados do recorte estão incompletos."

```

Backend: salvar e mostrar dados

```

1  nome_entrada_seguro = secure_filename(arquivo.filename)
2  caminho_upload = os.path.join(UPLOAD_FOLDER, nome_entrada_seguro)
3  arquivo.save(caminho_upload)
4
5  return f"""
6  <h1>Dados recebidos com sucesso</h1>
7  <p><strong>Imagem original salva em:</strong> {caminho_upload}</p>
8  <p><strong>Nome do novo arquivo:</strong> {nome_saida}</p>
9  <p><strong>X:</strong> {x}</p>
10 <p><strong>Y:</strong> {y}</p>
11 <p><strong>Largura:</strong> {w}</p>
12 <p><strong>Altura:</strong> {h}</p>
13 <p>Neste passo, o sistema ainda não recorta.</p>
14 """

```

Teste

Envie uma imagem com valores simples, por exemplo:

```

1  X = 0
2  Y = 0
3  Largura = 100
4  Altura = 100
5  Nome = recorte1.jpg

```

Checkpoint: se o navegador mostra os dados recebidos, o formulário e o backend já concordam sobre os nomes dos campos.

Passo 5 - Recortar de verdade com OpenCV

Desafio

Agora o backend deve abrir a imagem salva, validar os limites do recorte, cortar a matriz da imagem e salvar o novo arquivo.

Você vai praticar:

- `cv2.imread` para abrir imagem;
- `img.shape` para descobrir largura e altura;
- validação de coordenadas;
- fatiamento de matriz com `img[y:y+h, x:x+w]`;
- `cv2.imwrite` para salvar o recorte;
- `jsonify` para responder ao frontend.

Backend: imports e pastas

```
1 from flask import Flask, render_template, request, jsonify
2 from werkzeug.utils import secure_filename
3 import cv2
4 import os
5
6
7 UPLOAD_FOLDER = "uploads"
8 RECORTE_FOLDER = "recortes"
9
10 os.makedirs(UPLOAD_FOLDER, exist_ok=True)
11 os.makedirs(RECORTE_FOLDER, exist_ok=True)
```

Backend: começo da rota

```
1 @app.route("/recortar", methods=["POST"])
2 def recortar():
3     try:
4         if "imagem" not in request.files:
5             return jsonify({"erro": "Nenhuma imagem enviada"}), 400
6
7         arquivo = request.files["imagem"]
8         nome_saida = request.form.get("nome_saida", "").strip()
9         x = int(request.form.get("x", 0))
10        y = int(request.form.get("y", 0))
11        w = int(request.form.get("w", 0))
12        h = int(request.form.get("h", 0))
13
14        if not nome_saida:
15            return jsonify({"erro": "Nome do arquivo de saída não informado"}), 400
16
17        if w <= 0 or h <= 0:
18            return jsonify({"erro": "Largura e altura devem ser maiores que zero"}), 400
```

Backend: ler imagem e validar limites

```

1     nome_entrada_seguro = secure_filename(arquivo.filename)
2     caminho_upload = os.path.join(UPLOAD_FOLDER, nome_entrada_seguro)
3     arquivo.save(caminho_upload)
4
5     img = cv2.imread(caminho_upload)
6     if img is None:
7         return jsonify({"erro": "Não foi possível ler a imagem"}), 400
8
9     altura_img, largura_img = img.shape[:2]
10
11     if x < 0 or y < 0 or x + w > largura_img or y + h > altura_img:
12         return jsonify({"erro": "Área de recorte fora dos limites da imagem"}), 400

```

Backend: recortar e salvar

```

1     recorte = img[y:y+h, x:x+w]
2
3     nome_saida_seguro = secure_filename(nome_saida)
4     if not nome_saida_seguro.lower().endswith((".jpg", ".jpeg", ".png")):
5         nome_saida_seguro += ".jpg"
6
7     caminho_saida = os.path.join(RECORTE_FOLDER, nome_saida_seguro)
8
9     sucesso = cv2.imwrite(caminho_saida, recorte)
10    if not sucesso:
11        return jsonify({"erro": "Falha ao salvar o recorte"}), 500
12
13    return jsonify({
14        "mensagem": "Recorte salvo com sucesso",
15        "arquivo": caminho_saida
16    })
17
18    except ValueError:
19        return jsonify({"erro": "Valores de x, y, largura e altura devem ser
numéricos"}), 400
20    except Exception as e:
21        return jsonify({"erro": str(e)}), 500

```

Teste

Use uma imagem com tamanho conhecido. Primeiro tente um recorte pequeno:

```

1  X = 0
2  Y = 0
3  Largura = 100
4  Altura = 100

```

Depois tente provocar erro:

```

1  X = -10
2  Y = 0
3  Largura = 100
4  Altura = 100

```

Checkpoint: o sistema deve salvar recortes válidos e recusar áreas impossíveis.

Passo 6 - Responder sem trocar de página

Desafio

No passo anterior, o backend já responde JSON. Agora o frontend deve enviar o formulário com `fetch`, ler o JSON e mostrar o resultado na própria página.

O formulário não precisa mais ter `action` nem `method`, porque o JavaScript fará o envio.

HTML: formulário com id

```
1  <form id="formRecorte">
2    <div>
3      <label>Imagem:</label>
4      <input type="file" name="imagem" accept="image/*" required>
5    </div>
6
7    <div>
8      <label>X:</label>
9      <input type="number" name="x" required>
10   </div>
11
12   <div>
13     <label>Y:</label>
14     <input type="number" name="y" required>
15   </div>
16
17   <div>
18     <label>Largura:</label>
19     <input type="number" name="w" required>
20   </div>
21
22   <div>
23     <label>Altura:</label>
24     <input type="number" name="h" required>
25   </div>
26
27   <div>
28     <label>Nome do arquivo de saída:</label>
29     <input type="text" name="nome_saida" required>
30   </div>
31
32   <button type="submit">Enviar para recorte</button>
33 </form>
34
35 <p id="resultado"></p>
```

JavaScript: envio com fetch

```
1  <script>
2    const form = document.getElementById("formRecorte");
3    const resultado = document.getElementById("resultado");
4
5    form.addEventListener("submit", async (e) => {
6      e.preventDefault();
7
8      const dados = new FormData(form);
9
10     const resposta = await fetch("/recortar", {
```

```
1         method: "POST",
2         body: dados
3     });
4
5     const json = await resposta.json();
6
7     if (resposta.ok) {
8         resultado.textContent = `Sucesso: ${json.mensagem} - ${json.arquivo}`;
9     } else {
10         resultado.textContent = `Erro: ${json.erro}`;
11     }
12     });
13 </script>
```

Teste final

Envie uma imagem, informe as coordenadas e confira:

- a mensagem de sucesso aparece na página;
- a imagem original aparece em `uploads`;
- o recorte aparece em `recortes`;
- entradas inválidas retornam mensagens de erro.

Guia de investigação de erros

TemplateNotFound

Provável causa: o arquivo `index.html` não está dentro de `templates`.

Verifique:

```
1 recorte-imagem/  
2     recortes.py  
3     templates/  
4         index.html
```

404 Not Found

Provável causa: o endereço usado pelo formulário não existe no Flask.

Verifique se o valor de `action` no HTML combina com o `@app.route(...)` no Python.

405 Method Not Allowed

Provável causa: o formulário enviou `POST`, mas a rota não aceita `POST`.

Verifique:

```
1 @app.route("/recortar", methods=["POST"])
```

Nenhuma imagem enviada

Provável causa: o `name` do input não combina com o nome lido em `request.files`.

Verifique:

```
1 <input type="file" name="imagem">
```

```
1 arquivo = request.files["imagem"]
```

OpenCV não lê a imagem

Prováveis causas:

- o arquivo salvo não é uma imagem válida;
- o caminho está errado;
- o upload não foi salvo antes do `cv2.imread`.

Entrega

Ao terminar, mostre ao professor:

- o servidor rodando;
- uma imagem enviada;
- um recorte salvo;
- um caso de erro tratado corretamente;
- uma explicação curta do que acontece entre o clique no botão e o arquivo final salvo.

Desafios extras

Depois que o sistema funcionar, escolha um desafio:

- mostrar uma prévia da imagem antes do envio;
- criar um botão para limpar o formulário;
- mostrar as dimensões da imagem no navegador;
- impedir valores negativos já no HTML;
- listar os arquivos da pasta `recortes`;
- criar um link para baixar o recorte.